

Programmazione IPC

Antonio Pecchia

antonio.pecchia@unisannio.it

Dipartimento di Ingegneria
Laurea in Ingegneria Informatica
Sistemi Operativi



UNIVERSITÀ DEGLI STUDI
DEL SANNIO Benevento

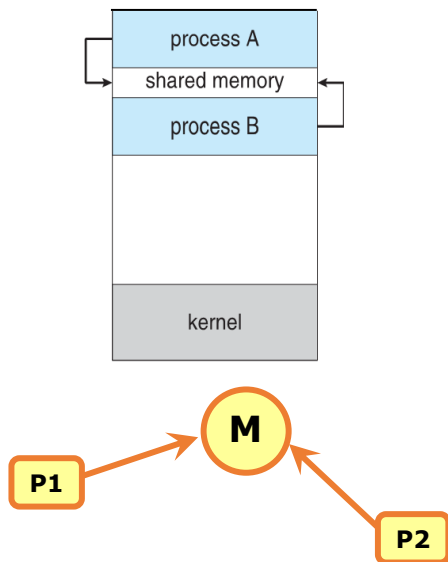
Inter-process communication*

- Comunicazione tra processi mediante le strutture dati rese disponibili dal kernel (**risorse IPC**):
 - memoria condivisa o *shared memory* (**shm**);
 - semafori (**sem**);
 - code di messaggi (**msg**).

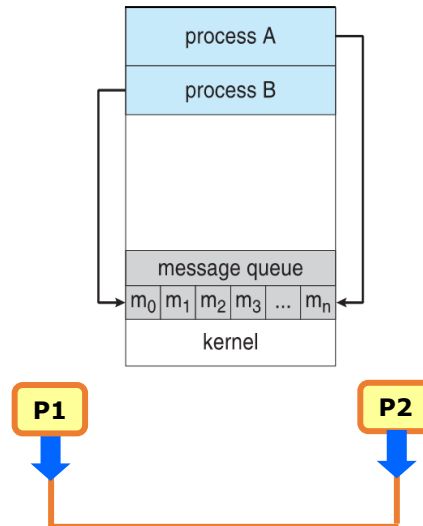
***NOTA:** la presentazione copre un insieme di funzionalità *base* fornite dalle primitive UNIX finalizzate all'esercitazione.

Inter-process communication

memoria condivisa



coda di messaggi



Chiave IPC

- ❑ Ogni risorsa IPC è identificata da un **valore univoco** nel sistema, denominata *chiave* (**IPC key**);
- ❑ Un *IPC key* può essere:
 - "cablata" nel codice;
 - generata dal SO tramite l'invocazione di **ftok***

*Non trattata nell'esercitazione.

Comandi `ipcs` ed `ipcrm`

- ❑ Comandi eseguibili da `shell`;
- ❑ `ipcs` `<-m|-s|-q>`:
 - `visualizza` tutte le strutture allocate (o solo shm, sem, msg) mostrandone anche l'identificatore e l'utente proprietario;
- ❑ `ipcrm` `<-m|-s|-q>` `<IPC_ID>`:
 - rimuove una data struttura, noto il suo identificatore.

Memoria condivisa

- ❑ I processi (a differenza dei thread) non possono condividere la memoria:
 - nemmeno i processi "parenti": il figlio, infatti, *eredita* una copia dei dati del padre;
 - se i due processi modificano quei dati dopo una fork, ciascun processo modifica la *propria* copia.
- ❑ Necessità di un strumento *dedicato*: `memoria condivisa`.

Memoria condivisa (utilizzo)

- ❑ creazione della SHM → `shmget`
 - ❑ collegamento alla SHM → `shmat`
 - ❑ uso della SHM;
 - ❑ scollegamento* della SHM; → `shmdt`
 - ❑ eliminazione* della SHM. → `shmctl` (flag `IPC_RMID`);
-
- ❑ **header file** da includere: `<sys/shm.h>`

*Non trattata nell'esercitazione (si ricorra al comando `ipcrm`).

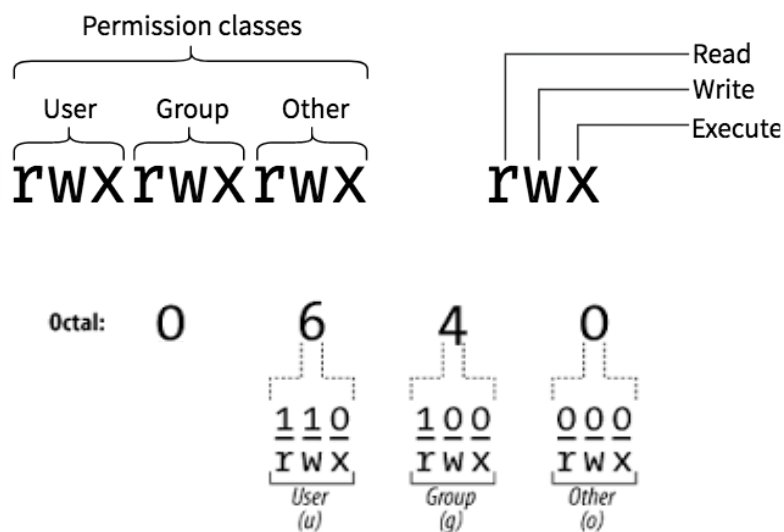
Creazione memoria condivisa*

```
int shmget(key_t key, int size, int flag)
```

- **key**: identifica la SHM in maniera univoca nel sistema (intero, o il risultato dell'invocazione `ftok`, o `IPC_PRIVATE`);
- **size**: dimensione (in byte) della SHM;
- **flag**: **modalità** di creazione e **permessi** di accesso (`IPC_CREAT`, `IPC_EXCL`, permessi);
- **restituisce** un identificatore numerico per la SHM in caso di successo (descrittore), -1 in caso contrario;

* <https://man7.org/linux/man-pages/man2/shmget.2.html>

Sidenote: codifica permessi



Collegamento memoria condivisa

```
void* shmat(int shmid, const void *shmaddr, int flag)
```

- **shmid**: identificatore SHM;
- **shmaddr**: indirizzo dell'area di memoria del processo chiamante al quale collegare la SHM (tipicamente impostato NULL);
- **flag**: opzioni (IPC_RDONLY per collegare in sola lettura, altrimenti read-write specificando 0);
- **Restituisce** l'indirizzo del segmento collegato, -1 in caso contrario.

Semafori: creazione*

```
int semget(key_t key, int nsems, int semflg);
```

- **key**: come nel caso della `shmget`;
 - **nsems**: **numero** di semafori → la `semget` consente di creare un **array di semafori**.
 - **semflg**: **modalità** di creazione e **permessi** (come nella `shmget`);
 - **restituisce** un identificatore numerico per l'array di semafori, -1 in caso contrario;
- **header file** da includere: `<sys/sem.h>`

* <https://man7.org/linux/man-pages/man2/semget.2.html>

Semafori: creazione ed inizializzazione

```
//...
//richiesta di 2 semafori ed inizializzazione
sem=semget(chiave,2,IPC_CREAT|0664);
//Inizializzazione dei due semafori
semctl(sem,0,SETVAL,val1);
semctl(sem,1,SETVAL,val2);
//...
```

id semaforo
(risultato `semget`)

↑
↑
indice dello **specifico**
semaforo nell'array

↑ valore iniziale desiderato

NOTA: possibilità di una *race condition* tra `semget` e `semctl`* se più processi accedono al semaforo con una chiave comune ed eseguono il codice di creazione-inizializzazione del semaforo.

*Link ad alcune soluzioni documentate alla race condition di cui sopra:

<https://cim.mcgill.ca/~jer/courses/os/lecnotes/ipc.html>

<http://www.deansys.com/doc/bgipc/semaphores.html>

Semafori: semop

`int semop(int semid, struct sembuf *sops, unsigned nsops)`

- applica a **semid** *N* operazioni (con $N=nsops$);
- ciascuna operazione è descritta tramite la struct **sembuf**;
- attributi di **sembuf**:

```
struct sembuf {  
    short sem_num; /* identifica l'indice del semaforo nell'array */  
    short sem_op;  /* operazione da applicare */  
    short sem_flg; /* modalità con cui è applicata l'operazione */  
};
```

L'insieme di operazioni inserite in una chiamata alla system call **semop** viene eseguito in modo **atomico**.

* <https://man7.org/linux/man-pages/man2/semop.2.html>

Semafori: semop

▣ Valori di **sem_op**:

< 0	equivale a P (si blocca se sem_val $\leq$ 0) Decrementa il semaforo della quantità ops.sem_op
= 0	In attesa che il valore del semaforo diventi 0
> 0	equivalente a V Incrementa il semaforo della quantità ops.sem_op

▣ Valori di **sem_flg***:

IPC_NOWAIT Per realizzare **P** e **V** non bloccanti
(comodo per realizzare *polling*)

SEM_UNDO Ripristina il vecchio valore quando termina

*Non trattati nell'esercitazione.

Semafori: semop

□ Valori di `sem_op`:

< 0	equivale a P (si blocca se <code>sem_val ≤ 0</code>) Decrementa il semaforo della quantità <code>ops.sem_op</code>
= 0	In attesa che il valore del semaforo diventi 0
> 0	equivalente a V Incrementa il semaforo della quantità <code>ops.sem_op</code>

□ Valori di `sem_flg`:

IPC_NOWAIT	Per realizzare P e V non bloccanti (comodo per realizzare <i>polling</i>)
SEM_UNDO	Ripristina il vecchio valore quando termina

Implementazione wait/signal tramite semop

```
void sem_wait (int id_sem, int numsem){  
  
    struct sembuf sem_buf;  
  
    sem_buf.sem_num=numsem;  
    sem_buf.sem_op=-1;  
    sem_buf.sem_flg=0;  
  
    semop(id_sem,&sem_buf,1);  
}
```

`wait` sul semaforo con indice numsem dell'array id_sem;

← in sembuf si indica numsem e si imposta sem_op=-1

← invocazione di `semop`

```
void sem_signal (int id_sem, int numsem){  
  
    struct sembuf sem_buf;  
  
    sem_buf.sem_num=numsem;  
    sem_buf.sem_op=1;  
    sem_buf.sem_flg=0;  
  
    semop(id_sem,&sem_buf,1);  
}
```

`signal` sul semaforo con indice numsem dell'array id_sem;

← in sembuf si indica numsem e si imposta sem_op=1

← invocazione di `semop`

NOTE RELATIVE ALL'ESERCITAZIONE

Informazioni generali

- Lo Studente scarichi il file `skeleton.c` dall'area didattica:
 - il file contiene le implementazioni di `wait` e `signal` realizzate tramite `semop` e `sembuf`;
 - **pre-requisiti**: si assume che lo Studente abbia familiarità di utilizzo della shell `bash` e gli strumenti a riga di comando `vi` e `gcc` trattati dalle precedenti lezioni pratiche;
 - in aula sono stati prodotti 2 file:
 - `mutex.c` (*mutua esclusione su un intero condiviso*);
 - `prodcons.c` (*produttore-consumatore su un buffer di N interi a gestione LIFO*).

Buffer di memoria condiviso*

- Esercizio **mutua esclusione**, un singolo intero:

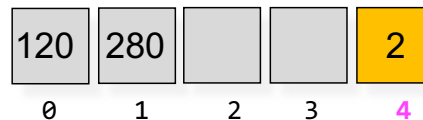
```
int shm_id = shmget( 0xaa, 4, IPC_CREAT | 0666 );
```

- Esercizio **produttore-consumatore**, N interi:

- spazio per **N interi + 1**, dove l'ultimo intero memorizza l'indice della prima posizione libera (*più a destra*) in cui produrre;
- il buffer ha logica LIFO (last-in, first-out):

```
int shm_id = shmget( 0xaa, 4*N+4, IPC_CREAT | 0666 );
```

- esempio N=4:



*Lo Studente non dimentichi l'invocazione di **shmat** dopo **shmget**

Buffer condiviso

- Funzione **produci**:

```
void produci( int* buf, int size, int val ){  
    int index = buf[size];  
  
    buf[index] = val;  
    printf("prodotto %d in indice %d\n", val, index );  
  
    index=index+1;  
    buf[size]=index;  
}
```

- Funzione **consuma**:

```
int consuma( int* buf, int size ){  
  
    int val=0;  
    int index = buf[size]-1;  
  
    val = buf[index];  
    printf("consumato %d da indice %d\n", val, index );  
    buf[size]=index;  
  
    return val;  
}
```

NOTA: **produci-consuma** non prevedono "particolari" controlli sul fatto che si possa eccedere la reale capienza *N* del buffer; il tutto è demandato alla corretta gestione dei semafori.

Semafori (produttore-consumatore)

- Si consiglia l'uso della direttiva `#define` per poter identificare in maniera più agevole i singoli semafori all'interno dell'array di semafori.

```
#define MUTEX 0
#define EMPTY 1
#define FULL 2
```

- **Creazione-inizializzazione** dei semafori:

```
semid = semget ( 0x11, 3, IPC_CREAT|0666 );

semctl ( semid, MUTEX, SETVAL, 1 );
semctl ( semid, EMPTY, SETVAL, N );
semctl ( semid, FULL, SETVAL, 0 );
```

NOTA: lo Studente introduca i **controlli necessari** affinché l'inizializzazione avvenga, *una sola volta*, alla creazione dei semafori (si ricordi il comportamento indotto dall'uso, o meno, di `IPC_CREAT`).

Produttore con semafori

- Lo Studente implementi la parte **consumatore** sulla base di quanto indicato durante l'esercitazione;
- Codice **produttore**:

```
sem_wait(semid, EMPTY);
sem_wait(semid, MUTEX);

produci ( buffer, N, a );

sem_signal(semid, MUTEX);
sem_signal(semid, FULL);
```